Recursion

Good Evening ☺

Today's content → Function call tracing

→ Recursion

→ How to write a recursive code?

→ Tracing

→ TC/SC of recursive code } Next class }

Why recursion?

→ Used in Mergesort / Quicksort

→ Binary Tree / BST / Tries

→ Dynamic programming

→ Back tracking

→ Graphs

Function call tracing

```
int add(n,m){
    return n+m;
}
```

```
int mul(x,y){
    return x*y
}
```

```
int sub(x,y){
    return x-y;
}
```

```
main(){
    x=10 , y=20
    print(sub(mul(add(x,y),30),75));
}
```

825

900

sub(mul(add(x,y),30),75) = 825

30

mul(add(x,y),30) = 900

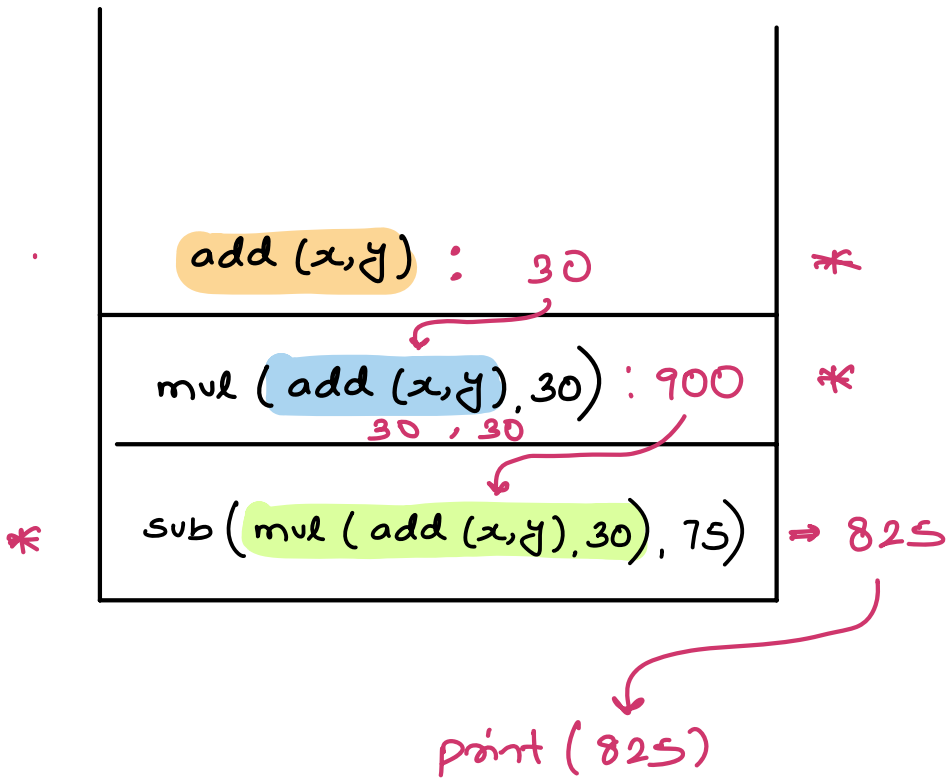
10 20

add(x,y) : = 30

Stack

↓

Last In First Out



Obs 1 → Whenever we call for function, it gets added at top

Obs 2 → When function returns, function will get removed from the top

Recursion → Repeating code / Reoccuring code

→ Function calling itself

→ Solving a problem using a subproblem

Q1. Sum of n natural number

$$sum(n) = \underbrace{1+2+3+\cdots+(n-1)}+n$$

$$sum(n) = \underbrace{sum(n-1)}+n$$

subproblem

$$sum(4) = \underbrace{1+2+3}+4$$

$sum(3)+4$

// sum(3) is a subproblem

* Steps to recursive code

01. Assumption / Expectation → Fix what should our function do or what is expected from the function

02. Main logic → Solving assumption using the subproblem

03. Base condition → Last valid input for which recursion has to stop

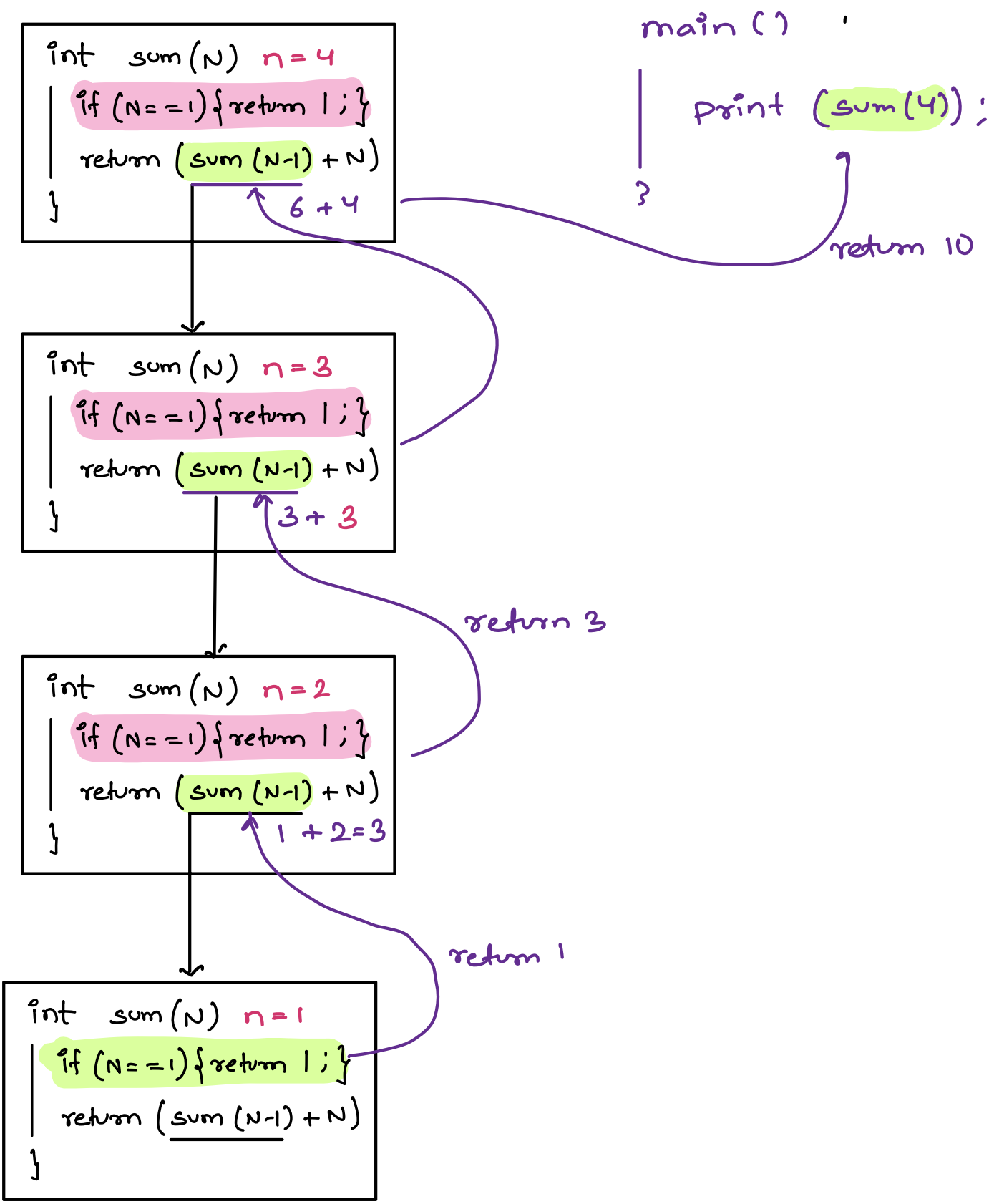
* Sum of n natural numbers (n>0)

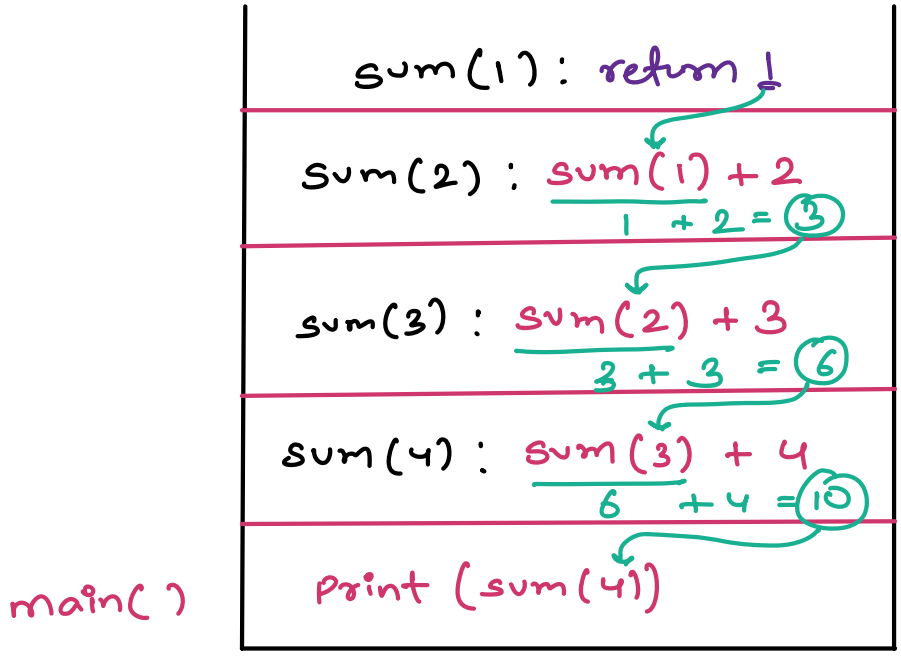
// Ass → Given n, calculate & return sum of n natural no.

```
int sum(n){
    if (n==1) return 1;

    return sum(n-1)+n;
}
```

Tracing


main ()
print (sum(4));
int sum(N) n=4
if (N==1){return 1;}
return (sum(N-1)+N)
6+4
return 10
int sum(N) n=3
if (N==1){return 1;}
return (sum(N-1)+N)
3+3
return 3
int sum(N) n=2
if (N==1){return 1;}
return (sum(N-1)+N)
1+2=3
return 1
int sum(N) n=1
if (N==1){return 1;}
return (sum(N-1)+N)



Without base condition

↳ Recursion will never stop

(a) TLE ⟶ X

(b) Memory Limit Exceeded / Stack overflow

Note :- Limit to function calls ≈ 10^5

Note :- If code is giving stack overflow, check for base condition

01. Can we write the base condition below recursive call?

First thing to write is base case

02. Factorial of a number ($n \geq 0$)

$$fact(3) = 3 * 2 * 1 = 6$$

$$fact(4) = 4 * 3 * 2 * 1 = 24$$

$$fact(n) = n * (n-1) * (n-2) * \cdots * 3 * 2 * 1$$

$$fact(4) = 4 * fact(3)$$

// Expectation → Given n, calculate & return n!

```
int fact(n)
{
    if (n==0) return 1;
    return n * fact(n-1);
}
```

0! = 1

Tracing → {TODO}

Fibonacii number ($N \geq 0$)

# Input	0	1	2	3	4	5	6	7	8
Fib()	0	1	1	2	3	5	8	13	21

fib( ) = sum of previous two fibonacii no.

Subproblem fib(n)

$$fib(7) = fib(6) + fib(5)$$

$$fib(80) = fib(79) + fib(78)$$

$$fib(n) = fib(n-1) + fib(n-2)$$

//Assumption → Calculate & return n^{th} fib number

```
int fib(n)
{
    if(n <= 1) return n;

    return fib(n-1) + fib(n-2);
}
```

Note :- Figure out the base case

↳ For what valid input, the main logic should not run

$fib(0) = fib(0-1) + fib(0-2)$

Invalid

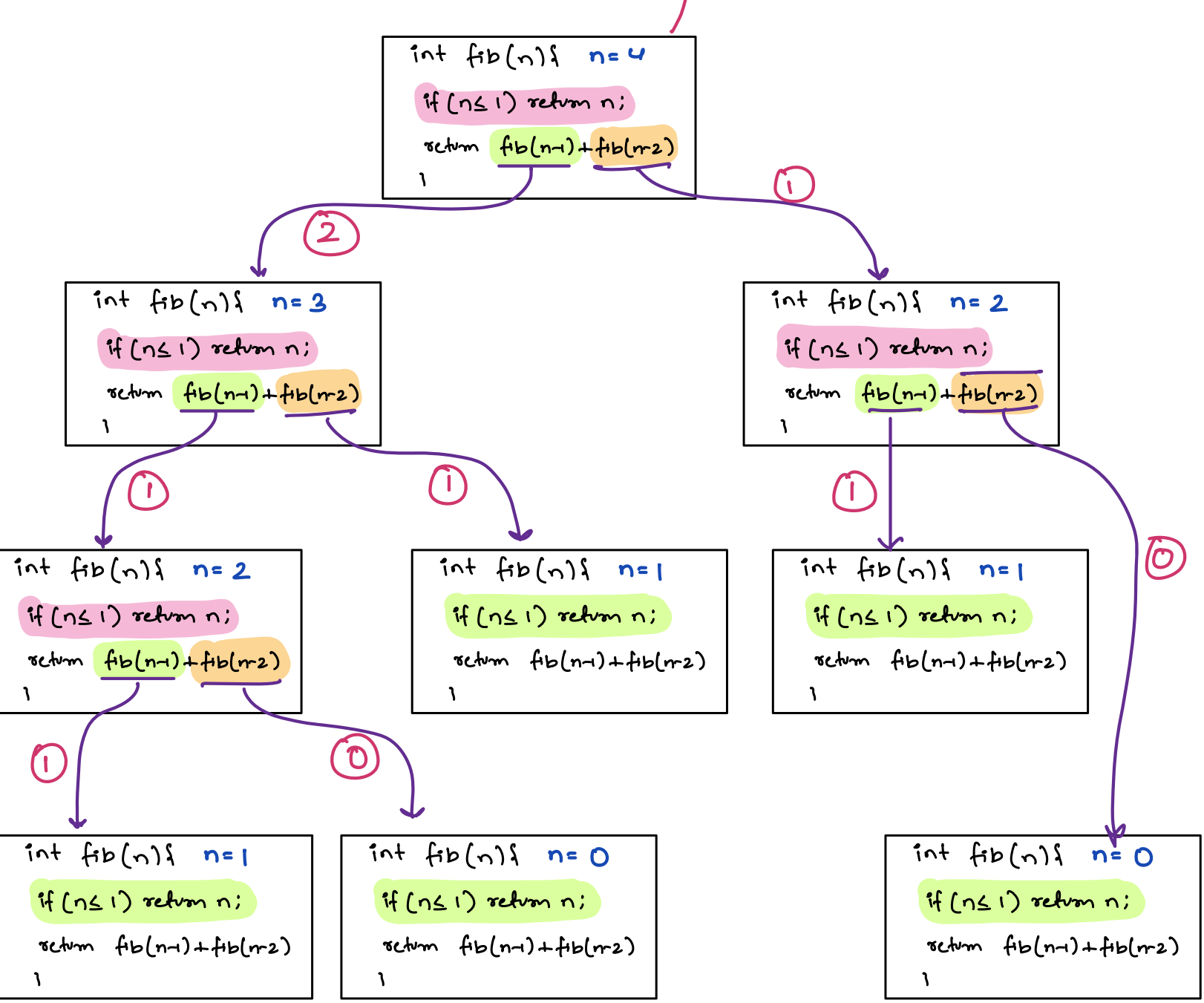
$fib(1) = fib(1-1) + fib(1-2)$

Invalid

Invalid, base condition for these two state

$fib(2) = fib(2-1) + fib(2-2)$

return 3 to main()

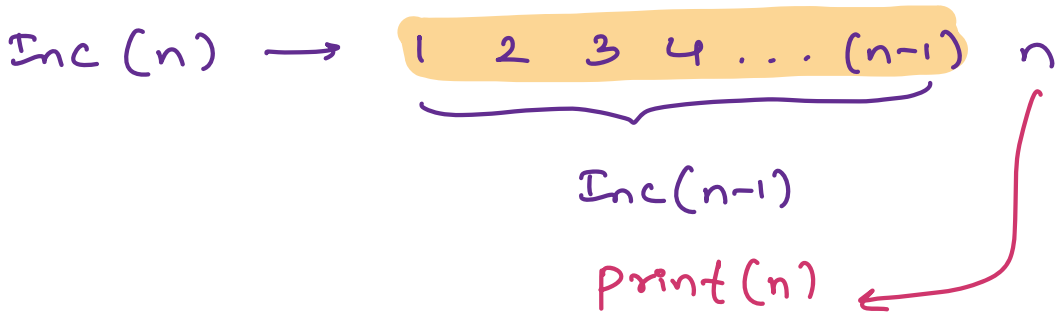


03. Given n, print all the numbers from 1 to N. in increasing order [n>0]

Inc (3) ⟶ 1 2 3

Inc (4) ⟶ 1 2 3 4

Inc (3) 4



// Assumption → Given n, print all the no. from 1 to n

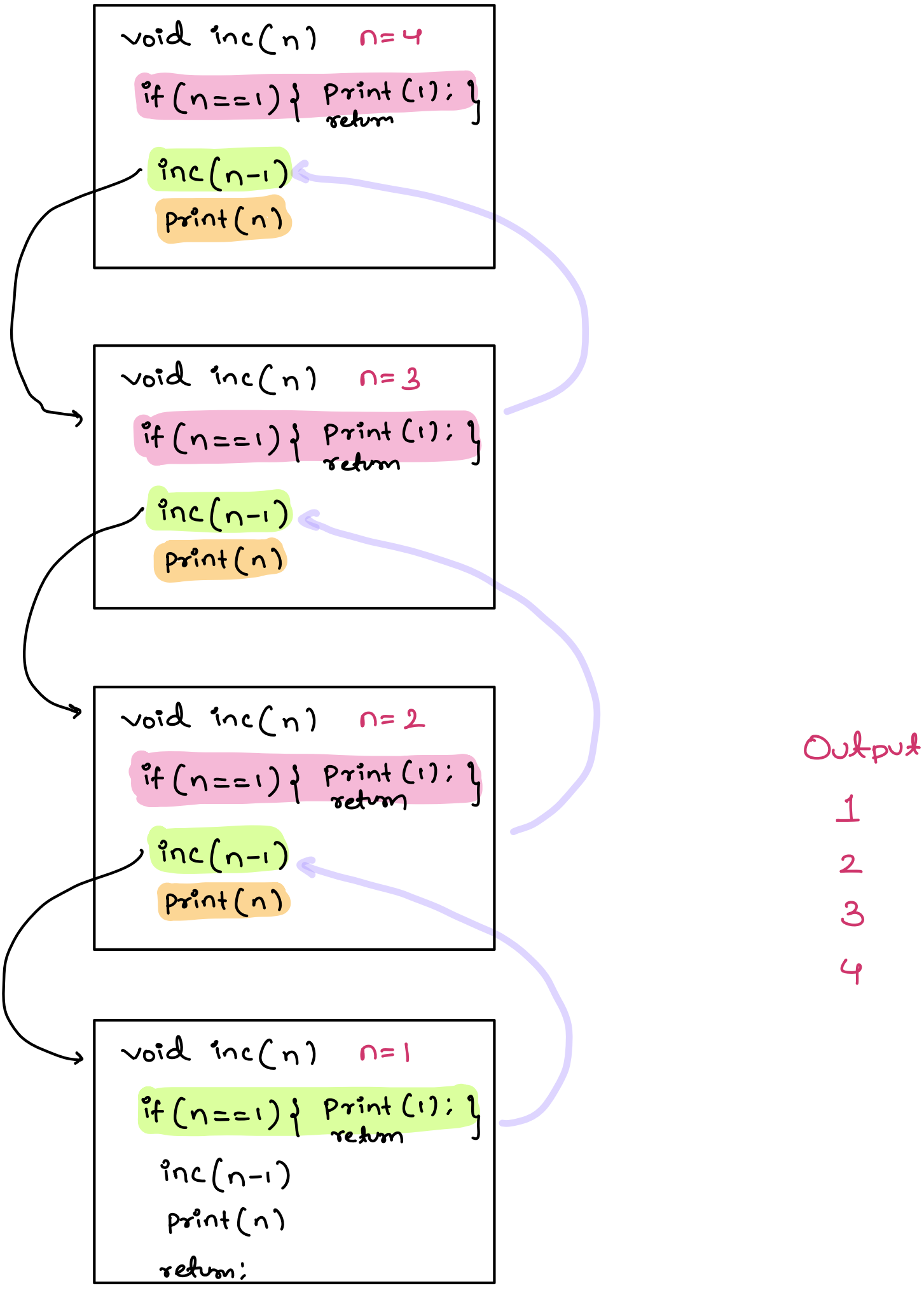
```
void inc(n)
{
    if (n == 1){
        print(1);
        return;
    }
    inc(n-1)
    print(n)
}
```

* Base condition
  ↳ The last valid input for which your main logic fail

Base case

Note :- Return statement means, the task of the function is complete, go back to your caller

Note :- Always return back from base case


void inc(n) n=4
if(n==1){ Print(1); return }
inc(n-1)
print(n)
void inc(n) n=3
if(n==1){ Print(1); return }
inc(n-1)
print(n)
void inc(n) n=2
if(n==1){ Print(1); return }
inc(n-1)
print(n)
void inc(n) n=1
if(n==1){ Print(1); return }
inc(n-1)
print(n)
return;

Output

1

2

3

4

Write the code

```
if (n>1) {
    inc(n-1)
    print(n)
}
3
```

TODO → { Write a function which prints the number in decreasing order

n = 4 Output → 4 3 2 1

n = 5 output → 5 4 3 2 1

05. Given a ch[ ], check if the given ch[] is palindrome for indexes [s < e]

ch[ ] = { a m a z o n } s = 2, e = 4
0 1 2 3 4 5

↓ False

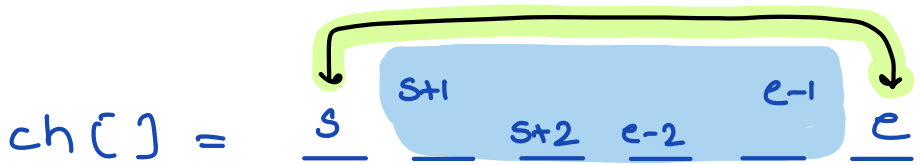
ch[ ] = { m a d a m } s = 1, e = 3
0 1 2 3 4

True

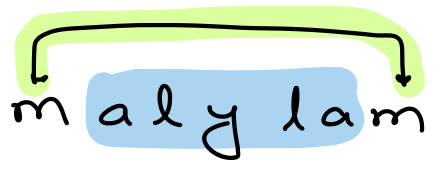
ch [ ] = { m a d a m } s = 0, e = 4
0 1 2 3 4

True

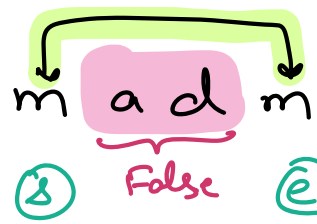
Assumption → Return if the ch array from s→e is a palindrome or not



Eg :-



→ True



→ False

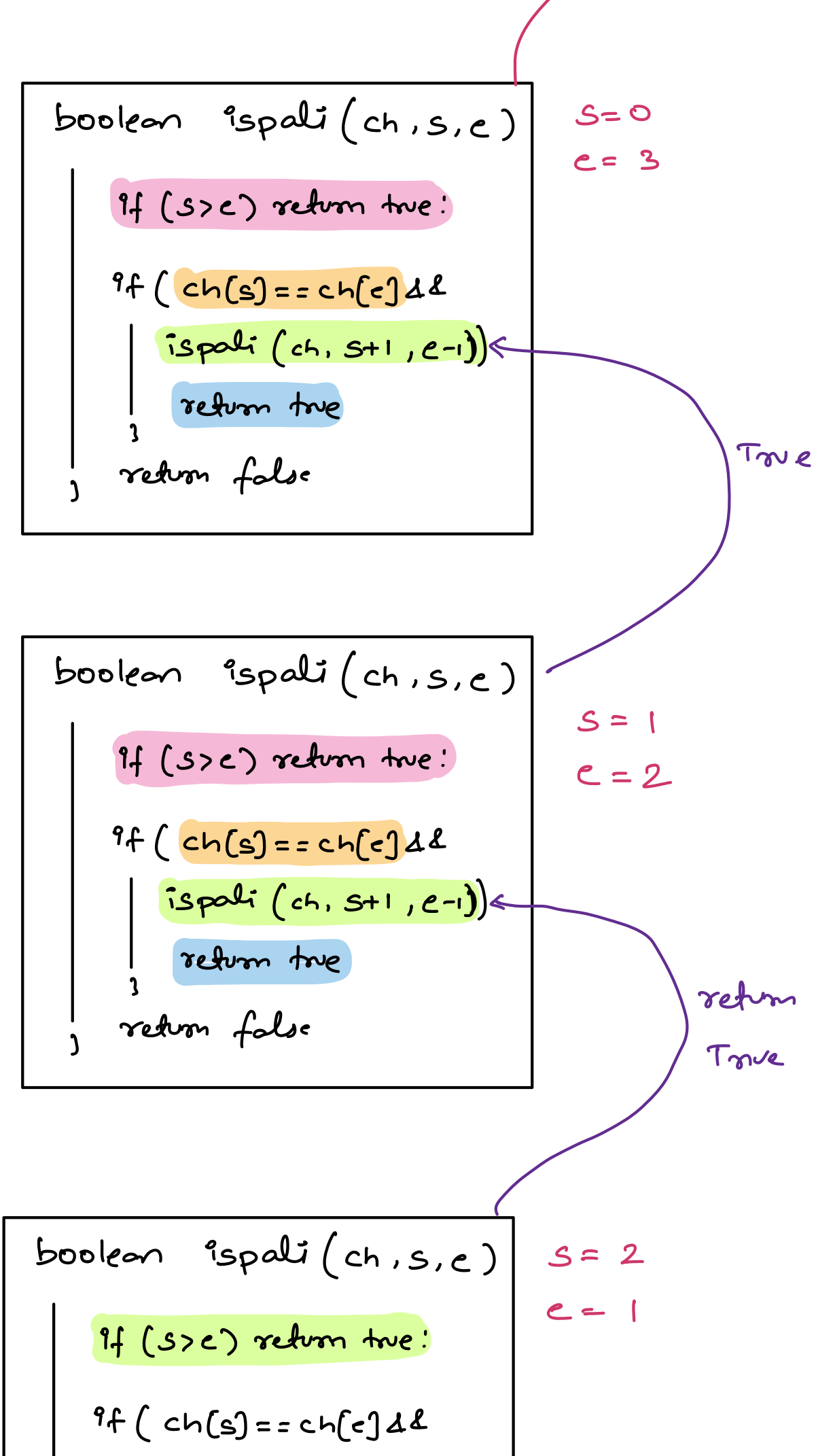
To return True → (s) & (e) must be same and the inner part must also be saying that i'm palindrome

```
boolean ispali ( ch[], int s, int e )
{
    if (s>e) return true;

    if ( ch[s] == ch[e]  && ispali (ch, s+1, e-1) == true)
    {
        return true;
    }
    return false;
}
```

```
main( ){
    ch[] = { n o o n }
             0 1 2 3
    s = 0
    e = 3

    print ( ispali(ch, s, e));
}
```


boolean ispali (ch, s, e)
if (s>e) return true:
if (ch[s] == ch[e] &&
ispali (ch, s+1, e-1))
return true
}
return false
}
s=0
e= 3
True
boolean ispali (ch, s, e)
if (s>e) return true:
if (ch[s] == ch[e] &&
ispali (ch, s+1, e-1))
return true
}
return false
}
s = 1
e = 2
return
True
boolean ispali (ch, s, e)
if (s>e) return true:
if (ch[s] == ch[e] &&
s = 2
e = 1

```
        ispali (ch, s+1, e-1))
        return true
    }
    return false
}
```

[s < e]

s = 0 e = 0 x

if (C_1 fails or C_2 fails)

return false